MessageFoundry — Deployment & Network Exposure Guide

Early Access · MessageFoundry v0.3.2 · July 2026

This is the consolidated reference for **how every MessageFoundry network channel binds, whether it supports TLS, and how it is authenticated and gated** — the artifact behind the v0.1 "native off-loopback TLS" gate (Gate #4). If you are about to expose the engine beyond `127.0.0.1`, read the checklist and the channel matrix first.

Design rationale for the off-loopback posture is in `adr/0002-phase2-transport-security-and-strong-auth.md`; PHI-in-transit context is in `PHI.md` §4; clustering topology is in `CLUSTERING.md`. For the host-level antivirus exclusions and Windows Firewall rules the engine needs, see `ANTIVIRUS-FIREWALL.md`.

On-premises by default

MessageFoundry runs **on-premises** and binds **loopback (`127.0.0.1`) by default** — the engine API (`[security].local_access_only = true`), and every inbound listener via `[inbound].bind_host`. In that posture **no MessageFoundry listener is reachable off-host**.

That is a claim about listeners only — it is not a claim that nothing PHI-bearing crosses a wire. Outbound delivery is **unaffected by the bind posture**: a loopback-bound engine still dials every configured destination (MLLP, REST/SOAP, FHIR, DICOMweb, SFTP/FTPS, SMTP, Direct, a customer database), still performs `db_lookup` / `fhir_lookup` reads, and still forwards syslog/SIEM logs and webhook alerts — all off-box, and those delivery hops carry PHI. The outbound controls below (egress allow-lists, the cleartext-hop rules, per-connection TLS) apply to a loopback deployment too.

Everything else below is about what changes when you deliberately bind a *channel* to a routable address.

Fail-closed rule (ADR 0002 §0): a non-loopback **API** bind is *refused at startup* unless TLS is configured (or an upstream TLS terminator is trusted), and every inbound **listen** type — MLLP, HTTP, DICOM C-STORE SCP, raw TCP/X12 — is refused off-loopback without TLS at wiring time.

The cleartext-bind escapes are clamped shut on the shipped posture (ADR 0148, ADR 0092 decision 2). `serve --allow-insecure-bind` — and its config twin `[security].require_encryption_for_remote = false` — only warn-and-cross while the instance is **not** enforcing-PHI. `[security].enforcement` defaults `enforce`, and all three built-in environment names (`dev`, `staging`, `prod`) now derive `data_class = phi`, so a **stock instance refuses the cleartext bind even with the flag**. Crossing it is a deliberate, recorded loosening: set `[security].enforcement = warn`, or declare the box synthetic with `[security].handles_real_patient_data = false`. Neither is a supported production setting.

Container deployment (Docker / Kubernetes)

The **headless engine** ships as an OCI image (ADR 0017 "container fast-follow") — a complement to the Windows-service/NSSM path (`SERVICE.md`), not a replacement. A container is just another way to bind off-loopback, so it inherits the controls above unchanged. The operator console is the **browser web console the engine serves in-process at `/ui`** (ADR 0065), but its optional `messagefoundry-webconsole` wheel is **not in the image** — neither hash-locked profile installs it, so a stock container soft-degrades to a **JSON-API-only serve** (with a warning) until you add the wheel to a derived image. Full build/run/ops guide: `../docker/README.md`; the scoping analysis is in `CONTAINER-EXPOSURE-EVALUATION.md`.

Container-specific essentials (all detailed in `../docker/README.md`):

- **Two variants:** slim default (core + SQLite) and `-sqlserver` (adds the OS-level MS ODBC Driver 18 for the SQL Server store / `db_lookup`). Non-root uid 10001; read-only root fs; per-profile hash-locked deps.
- **Config is executed code:** mount it owned by **uid 10001** and not group/world-writable, or — the robust path, and the only clean one on Kubernetes — **bake it into a derived image** (`FROM messagefoundry; COPY --chown=10001:10001 config /config`).
- **Store volume must persist** (named volume / PVC, never the ephemeral layer) or the at-least-once invariant is void across a restart; enable the at-rest cipher (`MEFOR_STORE_ENCRYPTION_KEY` + `MEFOR_STORE_REQUIRE_ENCRYPTION=true`).

- **Exposure:** Topology A (in-process TLS, the default) or Topology B (reverse-proxy / same-pod sidecar). Reaching a published port means binding off-loopback in the container, so the bind guard requires TLS — exactly as on bare metal. Every listen type is guarded (MLLP, HTTP, the DICOM SCP, raw TCP/X12); raw TCP/X12 have no TLS to enable, so publishing those ports means firewalling/segmenting them instead.
- **Signals:** PID 1 is `tini`; `SIGTERM` → graceful `engine.stop()`. Allow a stop grace of ≥30s.

Trust boundary — inside your organization's private network

MessageFoundry's supported deployment is *inside a single healthcare organization's private, trusted network* (on-prem or the org's private cloud), behind that org's perimeter controls — firewall, network segmentation, VPN/NAC. It is *never* placed directly on the public internet. This is the model every clinical interface engine assumes; state it explicitly in your own runbook, because it is the assumption every control here depends on and the first thing a security reviewer will ask for.

"Inside the network" is a statement about the **trust boundary**, *not* about which interface the engine binds. Three planes sit at different exposure levels:

Plane	What it is	Where it binds	Posture
Management	web console (`/ui`) / IDE → engine API	loopback by default (or a restricted management subnet)	required auth + RBAC + full audit; smallest surface — keep it off general-user VLANs
Data	inbound feeds you *receive* (MLLP, TCP/X12, DB-poll)	the **internal network interface** — feeds come from other systems on your LAN, not `127.0.0.1`	**TLS on the wire** (enable MLLP-over-TLS) + the `[egress]` /ingress allow-lists + your network segmentation. PHI must not cross the LAN in cleartext
Inbound web service	a partner *calls into* MEFOR (`Http()` source)	its own connector-owned socket	built (ADR 0023) — per-connection TLS + opt-in mTLS + IP allow-list, **no bearer/basic partner auth**; see the caveat below

The **management plane** is what you keep most contained; the **data plane is network-bound in any real install** (an EHR's MLLP feed is not on localhost) — which is exactly why MLLP-over-TLS and the fail-closed bind-guard exist. With TLS enabled on the data plane, PHI never crosses the LAN in cleartext.

Off-loopback security controls — delegate to your environment (and write it down)

Because the trust boundary is your private network, the controls that only become material once the engine leaves loopback are **satisfied by your organization's existing infrastructure** — *provided you document the delegation* and turn on the engine-side floor. This is an accepted way to meet the deployment-conditional OWASP ASVS items (tracked in the ASVS L3 remediation plan, an internal security-posture document not published in this repository):

Control (ASVS)	Delegate to your environment	Or build into the engine
Transport encryption (12.x)	— *enable* the shipped native API/WSS TLS + MLLP-over-TLS	already built (Gate #4)
MFA / multi-layer admin (6.3.3 / 8.4.2)	your **directory (AD / Entra)** — healthcare orgs are now *required* to enforce MFA there; MEFOR authenticates against it (see note below)	**native TOTP MFA is built and on by default** (ADR 0002 WP-14) — RFC 6238 for local accounts, `[security].require_mfa = true` with `require_mfa_scope = "every_local_account"` + the step-up gate; AD/Entra MFA stays delegated
TLS client-cert / mTLS (12.3.5)	your **PKI**; MF's API mTLS is built (`tls_client_ca_file` , opt-in)	enable mTLS + a console client cert
Certificate revocation (12.1.4)	your **proxy / PKI** (OCSP/CRL at the terminator)	document the delegation, or add OCSP/CRL to the TLS contexts
Off-box log shipping (16.4.3)	forward the audit + operational logs to your **SIEM/syslog**	**built** — `[logging].forward_*` ships operational logs + PHI-redacted audit rows to a syslog/SIEM collector, over **native TLS** with `forward_protocol = "tls"` (RFC 5425, ADR 0080; port 6514) (residual: the transport **default** is UDP, so set TLS explicitly or front the collector with a local TLS-forwarding agent)

Scoring caveat. That plan's **per-cell** scoring still reflects the pre-collapse posture columns: ADR 0148 collapsed deployment scoring to `{loopback, off-loopback}`, but the per-cell re-score and the owner's re-signature are an owner act and remain **pending**.

Write the delegation into your deployment runbook. "We run MEFOR inside our network behind \<perimeter / IdP / PKI / SIEM>" is what turns these from open gaps into *addressed-by-environment* — for your own risk posture and for any ASVS-scoped review.

On MFA specifically. Your **directory (AD / Entra) is the identity provider** and enforces MFA per your policy — which healthcare organizations are now **required** to do — so MEFOR does not re-implement it. One accuracy point for a security reviewer: a back-channel **LDAP simple-bind validates the password but does not itself prompt the second factor**, so MFA applies through **Kerberos / Windows SSO** (the workstation logon was already MFA'd), your **Conditional Access on a federated-SSO front**, or an **MFA-terminating reverse proxy**. Local accounts now have a **native second factor** — RFC 6238 TOTP (`[security].require_mfa`, WP-14) — **on by default for every local account**, so leave it on; AD / SSO remains preferable where your IdP already enforces and manages MFA centrally.

Caveat — accepting inbound web-service calls

If you intend to use MEFOR to **accept** web-service calls (a partner POSTs *into* the engine), the inbound listener **is built** — `Http(port=...)`, a connector-owned HTTP/1.1 receiver (ADR 0023) that answers `202 Accepted` once the raw body is durably committed to the ingress stage. It is a **distinct network surface even inside your LAN**, with its own bind/port in the connector layer, **separate** from the management API, and it does not inherit the API's auth: harden it deliberately.

- **Built:** per-connection TLS (`tls = true` + `tls_cert_file` / `tls_key_file`), opt-in **mTLS** via `tls_ca_file`, a per-connection `source_ip_allowlist`, DoS caps (`max_connections`, `receive_timeout`, `max_body_bytes`, `max_header_bytes`), and the off-loopback exposed-gate (`check_http_tls_exposure`).
- **Not built:** any *application*-layer partner authentication — there is no bearer/basic credential check on the listener, so **mTLS or the IP allow-list is the partner authentication**, or you front it with an authenticating reverse proxy. The synchronous downstream-reply (SOAP-envelope) path is also a defined ADR 0013 follow-on, not built: the first slice is respond-with-receipt only.

High availability — clients reach the engine through a floating VIP / L4 LB

MessageFoundry's HA model is **active-passive clustering** (N engine processes against one shared server DB; one leader runs the graph, the rest are warm standbys) — full setup in CLUSTERING.md. It changes the network picture in one way that belongs in your deployment plan:

- **Only the primary binds the inbound listener ports.** So senders must reach "the engine" through an operator-provided **floating VIP / L4 load balancer**, not a fixed node. Use **one VIP per inbound port with a TCP-connect health check on that port** — the check passes only on the primary, so the VIP lands inbound traffic on it and follows the primary across a failover. MLLP/TCP senders see a connection drop and reconnect through the VIP (make partners reconnect-on-drop).
- **The engine API is up on every node** (it's a control/read plane over the shared DB), so an API VIP can health-check the unauthenticated `GET /health` for liveness, or pin operations to the primary via `GET /cluster/status` (`role`) / `GET /cluster/nodes` (`leader_node_id`).
- **DB-tier HA is delegated to the database** (PostgreSQL streaming replication / SQL Server Always On) — MEFOR does not replicate the store itself.

MEFOR designs for the VIP and exposes the health-check/role endpoints, but **does not ship a load balancer** — you stand it up (keepalived, HAProxy, F5, a cloud NLB, ...). Single-node deployments need none of this.

Cloud / Kubernetes HA: for a multi-replica, Postgres-backed HA deployment on k8s — the copyable `replicas: 3` manifest, the L4-NLB-per-MLLP-port recipe (primary-only health check, no L7/HPA for MLLP), and the hybrid edge-relay topology — see `CLOUD-DEPLOYMENT.md`; for the cloud PHI / HIPAA posture (BAA, KMS, PrivateLink, region pinning), see `CLOUD-PHI-HIPAA.md` (both per ADR 0047).

Before you expose off-loopback

1. **API** — set `[security].local_access_only = false` + `[security].listen_address`, then either `[api].tls_cert_file` + `[api].tls_key_file` (in-process TLS) *or* `[api].tls_terminated_upstream = true` + `[api].trusted_proxies` (front it with a TLS terminator). Keep `[security].require_sign_in = true` (a non-loopback bind with sign-in disabled is refused, and no flag covers it). The legacy `[api].host` / `[auth].enabled` keys are **rejected at load** — they moved to `[security]` (ADR 0118).
2. **Web console (`/ui`)** — an off-loopback `/ui` **requires** in-process TLS or a declared terminator; `--allow-insecure-bind` does **not** cover the browser surface. The console is also on-by-default for *loopback* binds only: an exposed instance serves the JSON API alone unless you ask for it explicitly with `[security].serve_web_console = true`, and behind a declared terminator it additionally requires `[security].web_console_public_address` (the external https origin) or the serve refuses.

3. **MLLP inbound** — set `tls = true` + `tls_cert_file` / `tls_key_file` per connection. A non-loopback MLLP bind without `tls` is refused (`check_mllp_tls_exposure`). The DICOM SCP and inbound HTTP listeners have the same shape (`check_dimse_tls_exposure` / `check_http_tls_exposure`).
4. **Raw TCP / X12 inbound** — **no transport TLS exists** (these connectors are plaintext-only). They are, however, **exposed-gated** (since PR #558): a non-loopback raw-TCP/X12 bind is **refused at startup** (`check_tcp_tls_exposure`), parity with the MLLP/DICOM/HTTP guards. Because there is no TLS to enable, the only ways past the gate are a loopback bind or OS-level firewall/segmentation — on the shipped enforcing-PHI posture `serve --allow-insecure-bind` is clamped inert and the bind still refuses. See no-TLS hazards.
5. **Outbound connectors** — they default to verified TLS where the protocol supports it, and a **cleartext off-loopback hop is refused** on any `enforcement = enforce` instance — the hop authority no longer reads the instance's data label, so a *synthetic* box is refused too. Per-connection, the only honest way across is `cleartext_accepted = true` + `cleartext_reason` (ADR 0153: warn + audit at every construction, listed by `messagefoundry check` and `GET /security/posture`). Do **not** set `MEFOR_ALLOW_INSECURE_TLS` in production — it no longer influences a cleartext hop at all, and where it still applies it only weakens verification (see the escape hatch).
6. **Lock down egress** — populate the relevant `[egress].allowed_*` allow-lists so a transform can only send to approved destinations (see egress allow-lists). A PHI instance with *nothing* declared and deny-by-default off **refuses to start**.
7. **Off-box logs + MFA** — **both are built** and pair with off-loopback exposure: enable `[logging].forward_*` to ship logs + (PHI-redacted) audit to your SIEM (set `forward_protocol = "tls"` for the native RFC 5425 hop — the default is UDP — or front it with a local TLS agent), and leave `[security].require_mfa` on (it defaults on for every local account; AD/Entra MFA stays delegated to the IdP).

Channel × TLS posture matrix

Legend: **Bind** = default bind/connect posture · **TLS** = transport encryption support · **Auth** = authentication on the channel · **Egress gate** = the `[egress]` allow-list that confines it · **Off-loopback guarded?** = whether a non-loopback bind is refused without TLS.

Inbound (listeners — the engine binds a socket)

Channel	Bind default	TLS support	Auth	Ingress/egress gate	Off-loopback guarded?
Engine API (FastAPI/uvicorn)	`[security].local_access_only` = true → `127.0.0.1`	**Yes** — in-process via `tls_cert_file` / `tls_key_file`, *or* upstream via `tls_terminated_upstream` + `trusted_proxies`; `tls_min_version` (≥1.2); opt-in mTLS via `tls_client_ca_file`; HSTS over https	Bearer token + session RBAC (required)	— (auth-gated)	**Yes** — refused without TLS or a trusted terminator, and `--allow-insecure-bind` is clamped inert on an enforcing PHI instance (the default); also refused if sign-in is disabled on a non-loopback bind
MLLP source	`[inbound].bind_host` = `127.0.0.1`	**Yes** — per-connection opt-in `tls=true` + `tls_cert_file` / `tls_key_file`; opt-in mTLS via `tls_ca_file`; ≥TLS 1.2. **Plaintext by default**	None (MLLP has no app auth)	—	**Yes** — non-loopback plaintext refused (`check_mllp_tls_exposure`)
HTTP source (`Http()`, ADR 0023)	`[inbound].bind_host` = `127.0.0.1`	**Yes** — per-connection opt-in `tls=true` + `tls_cert_file` / `tls_key_file`; opt-in mTLS via `tls_ca_file`. **Plaintext by default**	mTLS client cert only — **no bearer/basic partner auth**	per-connection `source_ip_allowlist`	**Yes** — non-loopback plaintext refused (`check_http_tls_exposure`)
DICOM C-STORE SCP (`DICOM()`, ADR 0025)	`[inbound].bind_host` = `127.0.0.1`	**Yes** — per-connection opt-in `tls=true` + cert/key; opt-in mTLS via `tls_ca_file`. **Plaintext by default**	`calling_ae_allowlist` / `require_called_ae_title` / mTLS (DIMSE has no transport auth of its own)	per-connection `source_ip_allowlist`	**Yes** — non-loopback plaintext refused (`check_dimse_tls_exposure`), **and** a non-loopback SCP with *no* peer control (calling-AE allow-list, IP allow-list, or mTLS) is refused at construction

Channel	Bind default	TLS support	Auth	Ingress/egress gate	Off-loopback guarded?
Raw TCP source	`[inbound].bind_host` = `127.0.0.1`	**No** — plaintext only	None	—	**Yes** — non-loopback plaintext refused (`check_tcp_tls_exposure`, PR #558); no TLS to enable, so keep loopback / firewall-segment / proxy-terminate
X12 source (ISA/IEA framed)	`[inbound].bind_host` = `127.0.0.1`	**No** — plaintext only (same socket plumbing as raw TCP)	None	—	**Yes** — non-loopback plaintext refused (`check_tcp_tls_exposure`, PR #558); keep loopback / firewall-segment / proxy-terminate
File source	local filesystem	n/a (no network)	n/a	—	n/a
Database poll source	**binds no socket** — dials an operator-configured DB host: its own `server` (required) and `port` (default `1433`). **Not** the `[store]` database, and it inherits nothing from `[store]`	**Yes, per connection** — its own `encrypt` (default true) + `trust_server_certificate` (default false) on the default `dialect="sqlserver"`. On `dialect="generic"` TLS is the ODBC driver's own keyword and is **not** engine-enforced	its own `auth` / `username` / `password`	`[egress].allowed_db`	n/a (outbound DB connection)

Outbound (the engine dials a destination)

Channel	Connect	TLS support	Auth	Egress gate
MLLP destination	dials host:port	**Yes** — per-connection `tls=true`; `tls_verify=true` **default**; client-cert mTLS via `tls_cert_file` / `tls_key_file` + `tls_ca_file`; ≥TLS 1.2	peer HL7 ACK	`[egress].allowed_mllp`
Raw TCP destination	dials host:port	**No** — plaintext only	None	`[egress].allowed_tcp`
X12 destination	dials host:port	**No** — plaintext only	None (optional TA1)	`[egress].allowed_tcp`
REST destination	dials URL	**HTTPS by default** — `verify_tls=true` default (downgrade refused without the escape); cleartext-credential `http` refused; 3xx redirects refused	optional `Authorization` (Basic/Bearer), refused over plaintext	`[egress].allowed_http`
SOAP destination	dials URL	**HTTPS by default** — reuses the REST client + no-redirect opener; per-connection client-cert mTLS; ≥TLS 1.2 in the mTLS context	optional WS-Security `UsernameToken` (Nonce + Timestamp)	`[egress].allowed_http`
DATABASE destination	dials server:port	**Yes** — SQL Server `Encrypt=yes` **default**, `TrustServerCertificate=false` default (weakened only via the escape)	ODBC `sql` / `integrated` / `entra`	`[egress].allowed_db`
File destination	local filesystem	n/a (no network)	n/a	`[egress].allowed_file_dirs`
RemoteFile destination + source (SFTP / FTPS / FTP)	dials remote host	**Protocol-dependent** — **SFTP** encrypted (SSH host-key verify on by default); **FTPS** explicit TLS; **FTP** plaintext (credentials refused without the escape)	username/password or SSH key	`[egress].allowed_remote`

Above and beyond each row: an **off-loopback cleartext outbound hop is decided by one authority** for every connector (ADR 0092, amended by ADR 0153) — loopback ALLOW, then a per-connection `cleartext_accepted` + `cleartext_reason` WARN (logged + audited at every construction), then WARN while `[security].enforcement` is not `enforce`, else **REFUSE**. It no longer reads the instance's data label, and `MEFOR_ALLOW_INSECURE_TLS` no longer reaches it at all.

Internal

Channel	Transport	TLS
Inter-node cluster coordination (active-passive HA / Track B)	**Rides the shared `[store]` DB connection — no separate node-to-node socket.** Leadership lease, heartbeat (`last_seen`), and config-version bumps are reads/writes against cluster tables on the same pool.	**= the store DB connection's TLS** (`DbCoordinator` on the asyncpg pool for PostgreSQL, `SqlServerCoordinator` on the aioodbc pool for SQL Server). Encrypt the store connection and the cluster traffic is encrypted with it.
Store DB connection (PostgreSQL / SQL Server)	asyncpg / aioodbc pool	**Yes** — `[store].encrypt` (default true) + `[store].trust_server_certificate` (default false); weakened only via the escape

---

No-TLS channels — hazards

These channels have **no transport encryption at all** — there is no per-connection `tls` option as there is for MLLP:

- **Raw TCP source/destination** — plaintext, arbitrary framing.
- **X12 source/destination** — plaintext ISA/IEA-framed EDI interchanges.
- **Plain FTP** (RemoteFile `protocol=ftp`, as opposed to SFTP/FTPS) — cleartext protocol; credentials and file contents cross the wire in the clear (the connector refuses credentials over plain FTP unless the escape is set).

Deployment requirement: run these on **loopback only**, or behind a **TLS-terminating proxy / on a trusted, isolated network segment**. If PHI flows over one of them off-host without that protection, it is exposed in cleartext. A non-loopback raw-TCP/X12 **bind** is refused at startup (`check_tcp_tls_exposure` , PR #558) — but because these connectors have no TLS to enable, the gate's only passes are a loopback bind or OS-level firewall/segmentation (`serve --allow-insecure-bind` is clamped inert on the default enforcing-PHI posture); choosing one (and keeping PHI off the cleartext wire) is the **operator responsibility**. On the **outbound** side these are cleartext *hops*, so they are governed by the hop authority instead: off-loopback they **refuse** on an enforcing instance unless the connection declares `cleartext_accepted` + `cleartext_reason` . For raw TCP and X12 that declaration is **permanent, not transitional** — there is no `tls = true` for them to migrate to (ADR 0153 decision 4; TLS support for them is BACKLOG #311). Credentialed plain FTP is refused outright on an enforcing PHI instance (it puts the credential itself on the wire).

The `MEFOR_ALLOW_INSECURE_TLS` escape hatch

Several connectors **fail closed** on a weakened-TLS or cleartext-credential configuration unless the environment variable `MEFOR_ALLOW_INSECURE_TLS` is set. It exists for **dev / trusted-lab** use only. With it set, these otherwise-refused settings become permitted (each logs a loud warning):

- REST/SOAP `verify_tls = false` . *(Clamped.)*
- MLLP outbound `tls_verify = false` ; FTPS `tls_verify = false` . *(Clamped.)*
- DATABASE destination / store: `Encrypt=false` or `TrustServerCertificate=true` (SQL Server), `[store].trust_server_certificate=true` / `[store].encrypt=false` . *(Clamped.)*
- Plain-FTP credentials. *(Clamped.)*
- RemoteFile SFTP: accepting an unknown host key. *(Not clamped — the raw escape still applies.)*
- Cleartext SMTP submission on a **Direct** (S/MIME) destination. *(Not clamped; AUTH credentials over cleartext stay refused outright either way.)*
- The non-connection cells that have nowhere to carry a per-hop declaration: the `[logging]` syslog/SIEM forwarder and the API PHI-read serve hop *(both clamped)*, plus LDAPS, the webhook alert sink and the AI-broker endpoint *(raw escape)*.

Two limits worth stating plainly. *(a)* Since ADR 0153 this variable has been **unhooked from the cleartext-hop authority** — that decision no longer reads it, nor the instance's data label — so cleartext credentials over `http` , cleartext MLLP/DICOM/DICOMweb and the cleartext HTTP family are now governed only by a per-connection `cleartext_accepted` + `cleartext_reason` (warn + audit) or a loopback hop. (The engine also honours a `tls_hop_attested` hop — the opposite claim, "secure by other means", a silent ALLOW — but that field has **no authoring surface on a connection**: no factory parameter and no `connections.toml` key, so it is unreachable from config today. Refusal messages that suggest it are ahead of the code.) *(b)* Where it does still apply it is mostly **clamped** (ADR 0092 decision 2 / ADR 0148): it cannot relax a hop while

`[security].enforcement = enforce`, and for the MLLP/FTPS/plain-FTP and store-TLS cells the clamp additionally requires the instance to be PHI — which is also the default. Either way, on the shipped posture those cells are inert; the bullets marked *not clamped* are the exceptions that still honour the raw variable.

Never set `MEFOR_ALLOW_INSECURE_TLS` in production. Its presence is the single switch that turns the remaining fail-closed verification checks into best-effort.

Egress allow-lists

Outbound destinations are confined by per-protocol allow-lists in `[egress]` (`config/settings.py`). An **empty** list means unrestricted only while deny-by-default is off — which, on a PHI instance, it is not (see below). Once a list is **populated, it is fail-closed**: a destination of that type that does not resolve to a listed `host:port` makes the config **fail at load / reload / start** (validated *after* `env()` substitution, so dynamic addresses are checked against the resolved value).

Setting	Confines
`[egress].allowed_mllp`	MLLP destinations
`[egress].allowed_tcp`	raw TCP **and** X12 destinations
`[egress].allowed_http`	REST, SOAP, **FHIR**, **DICOMweb (STOW-RS)** destinations, the **SMART token endpoint**, and the read-only `fhir_lookup`
`[egress].allowed_db`	DATABASE destination + the DB poll source
`[egress].allowed_remote`	RemoteFile SFTP/FTPS/FTP (source + destination)
`[egress].allowed_file_dirs`	File destination directories

The alert sinks are *not* on this table and are not covered by `[egress]`. The webhook and SMTP *alert* sinks carry no PHI bodies and keep their **own** host allow-lists — `[alerts].webhook_allowed_hosts` and `[alerts].smtp_allowed_hosts`. Populate those separately; an `[egress].allowed_http` entry does nothing for a webhook alert.

For an off-loopback deployment, populate the lists you use so a transform cannot exfiltrate to an unapproved address. The **global deny-by-default toggle is built** — `[security].block_unlisted_outbound` — and on a **PHI instance the serve gate turns it on for you** unless you set it explicitly, so a transport whose `allowed_*` list is empty then refuses *every* destination of that type. Related refusal: a PHI instance with **no** allow-list populated *and* deny-by-default off has fully unrestricted egress and **refuses to start** under `[security].enforcement = enforce` (it warns at `warn`).

Bind-guard behavior (summary)

- **API** (`__main__.py`): a non-loopback bind is refused unless in-process TLS is configured, or `tls_terminated_upstream` + `trusted_proxies` are set; also refused if `[security].require_sign_in = false`, which no flag covers. Override (dev only): `serve --allow-insecure-bind` — **clamped inert on an enforcing PHI instance**, i.e. on the shipped default.
- **MLLP inbound** (`pipeline/wiring_runner.py`, `check_mllp_tls_exposure`): a non-loopback MLLP source without `tls=true` raises a `WiringError` at wiring time (before the engine starts). Override (dev only): `serve --allow-insecure-bind`, under the same clamp.
- **DICOM C-STORE SCP / HTTP / raw-TCP / X12 inbound** (same module): siblings of the MLLP guard — `check_dimse_tls_exposure`, `check_http_tls_exposure`, and `check_tcp_tls_exposure` (raw-TCP **and** X12, shipped in PR #558) — each refuses a non-loopback bind without TLS at wiring time. raw-TCP/X12 are plaintext-only, so for them the only passes are loopback or OS firewall/segmentation. So every inbound listen type is now exposed-gated.
- **The clamp, precisely** (ADR 0148, ADR 0092 decision 2): all four inbound gates and the API gate honour `--allow-insecure-bind` only while the instance is **not** (`enforcement = enforce` **and** PHI). Both halves are the default, so on a stock instance the flag changes nothing — the recorded loosening is `[security].enforcement = warn` or `[security].handles_real_patient_data = false`. These refusals also name `tls_hop_attested`, which the gates do read, but that field has no authoring surface on a connection today (see the escape hatch).
- **Browser console (`/ui`)**: an off-loopback `/ui` additionally requires in-process TLS or a declared terminator and is refused without one — `--allow-insecure-bind` does not cover it.

Maintenance: keep this matrix in sync with `transports/`, `config/settings.py` (`[security]` / `[egress]` / `[api]` / `[store]`), `config/tls_policy.py` (the hop authority), and the bind-guards. Cross-referenced from `PHI.md` §4, `CLUSTERING.md`, and ADRs 0002 / 0148 / 0153.
